

দিন 4

Linked List, Stack, Queue ও Deque

Pointer নিয়ে চিন্তা করা শেখো — ইন্টারভিউয়ারদের প্রিয় টপিক

আজ যা শিখবো

- Linked List: Node, append, traverse, insert, delete
- Array বনাম Linked List
- Stack (LIFO) ও Queue (FIFO)
- Deque — দুই দিক খোলা queue
- PHP-তে SplStack, SplQueue
- ৬টি গুরুত্বপূর্ণ কোডিং প্রশ্ন

কোর্স ভিডিও: ১৫-২১

দিন শেষে তুমি পারবে

- PHP-তে Node class দিয়ে linked list বানাতে ও ঘুরতে পারবে
- Linked list উল্টাতে পারবে (সবচেয়ে কমন প্রশ্ন)
- Fast-slow pointer দিয়ে মাঝখান আর cycle খুঁজতে পারবে
- Stack দিয়ে Valid Parentheses সলভ করতে পারবে

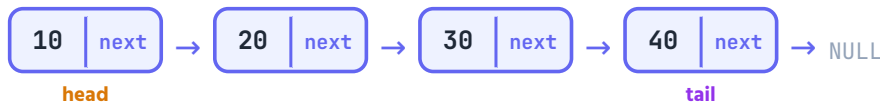
স্টাডি শিডিউল (প্রায় ৫-৬ ঘণ্টা)

১ ঘণ্টা	Linked list কনসেপ্ট + নিজে বানানো
১ ঘণ্টা	Reverse Linked List (৩ বার লেখো!)
১ ঘণ্টা	Middle + Cycle (fast-slow pointer)
৪৫ মিনিট	Merge Two Sorted Lists
১ ঘণ্টা	Stack/Queue কনসেপ্ট + Valid Parentheses + Queue using Stacks
৩০ মিনিট	খিওরি + চিট শিট

অধ্যায় ১: Linked List কী?

Linked list হলো **Node**-এর একটা শিকল। প্রতিটা node-এ দুইটা জিনিস থাকে: **val** (ভেটা) আর **next** (পরের node-এর ঠিকানা)। প্রথম node-কে বলে **head**, শেষ node-এর next হলো **NULL**।

head থেকে শুরু, NULL-এ শেষ



বাস্তব উদাহরণ: গুপ্তধন খোঁজার খেলা

প্রতিটা চিরকুটে একটা ধাঁধা আর পরের চিরকুট কোথায় আছে তার ঠিকানা লেখা। ৫ নম্বর চিরকুটে সরাসরি যেতে পারবে না — ১ থেকে শুরু করে একে একে যেতে হবে। কিন্তু মাঝে নতুন চিরকুট ঢোকানো সহজ: শুধু আগের চিরকুটের ঠিকানা বদলে দাও।

Array বনাম Linked List

	Array	Linked List
মেমোরিতে	পাশাপাশি (contiguous)	ছড়িয়ে ছিটিয়ে, pointer দিয়ে জোড়া
i-তম element access	O(1)	O(n) — head থেকে হাঁটতে হয়
শুরুতে insert/delete	O(n) — সবাইকে সরাতে হয়	O(1) — শুধু head বদলাও
মাঝে insert (node জানা থাকলে)	O(n)	O(1)
সাইজ	Fixed (traditional)	ইচ্ছামতো বাড়ে-কমে
বাড়তি মেমোরি	নেই	প্রতিটা node-এ next pointer

Linked List-এর ধরন

- **Singly:** প্রতিটা node শুধু পরেরটা চেনে (next)। সবচেয়ে কমন।
- **Doubly:** next আর prev দুটোই আছে — দুই দিকে যাওয়া যায়। Browser-এর back/forward।
- **Circular:** শেষ node আবার head-কে দেখায়। যেমন গানের playlist repeat।

কোথায় ব্যবহার হয়?

- Browser history (doubly)
- Music playlist (next / previous গান)
- Undo ফিচার
- LRU Cache (hash map + doubly linked list)
- Stack আর Queue বানাতে

প্রশ্ন 4.1

Build a Linked List (Node, Append, Traverse)

PHP-তে নিজে linked list তৈরি করো

Easy

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

ইন্টারভিউতে অনেক সময় বলে "linked list-এর Node class আর append/delete লেখো।" এটা না পারলে কোনো linked list প্রশ্নই সলভ করা যাবে না।

সমস্যাটা কী?

একটা `ListNode` class বানাও। তারপর এই অপারেশনগুলো লেখো: শেষে যোগ (append), শুরুতে যোগ (prepend), সব মান প্রিন্ট (traverse), একটা মান খুঁজে মুছে ফেলা (delete)।

উদাহরণ

```

append(1), append(2), append(3) → 1 → 2 → 3 → NULL
prepend(0)                        → 0 → 1 → 2 → 3 → NULL
delete(2)                        → 0 → 1 → 3 → NULL

```

LinkedList class — সম্পূর্ণ কোড

1. **append:** head null হলে নতুন node-ই head; নইলে শেষ পর্যন্ত হেঁটে শেষ node-এর next-এ বসাও
2. **prepend:** নতুন node-এর next = পুরনো head; head = নতুন node
3. **traverse:** curr = head থেকে curr = curr->next যতক্ষণ না null
4. **delete:** head-ই মুছতে হলে head = head->next; নইলে যার next-এ টার্গেট, তার next = next->next

PHP

```

class ListNode {
    public $val;
    public ?ListNode $next;
    public function __construct($val = 0, ?ListNode $next = null) {
        $this->val = $val;
        $this->next = $next;
    }
}

class LinkedList {
    public ?ListNode $head = null;

    public function append($val): void {           // O(n)
        $node = new ListNode($val);
        if ($this->head === null) { $this->head = $node; return; }
        $curr = $this->head;
        while ($curr->next !== null) {             // শেষ node খোঁজো
            $curr = $curr->next;
        }
        $curr->next = $node;
    }

    public function prepend($val): void {          // O(1)
        $this->head = new ListNode($val, $this->head);
    }

    public function delete($val): void {           // O(n)
        if ($this->head === null) return;
        if ($this->head->val === $val) {            // head-ই মুছতে হবে
            $this->head = $this->head->next;
            return;
        }
        $curr = $this->head;
        while ($curr->next !== null && $curr->next->val !== $val) {
            $curr = $curr->next;
        }
        if ($curr->next !== null) {
            $curr->next = $curr->next->next;        // মাঝের node বাদ
        }
    }

    public function printList(): void {            // traverse O(n)
        $curr = $this->head;
        while ($curr !== null) {
            echo $curr->val . " -> ";
            $curr = $curr->next;
        }
        echo "NULL\n";
    }
}

$list = new LinkedList();
$list->append(1); $list->append(2); $list->append(3);
$list->printList();           // 1 -> 2 -> 3 -> NULL
$list->prepend(0);
$list->printList();           // 0 -> 1 -> 2 -> 3 -> NULL

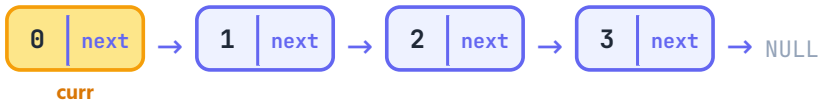
```

```
$list->delete(2);
$list->printList();           // 0 -> 1 -> 3 -> NULL
```

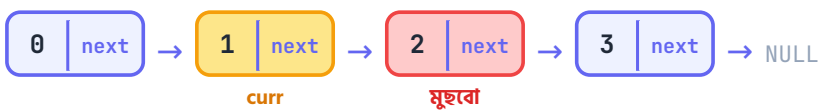
Time: append $O(n)$, prepend $O(1)$, delete $O(n)$ Space: $O(1)$ প্রতি অপারেশনappend $O(1)$ করতে চাইলে একটা `$tail` pointer রাখো — শেষ node সবসময় জানা থাকবে।

ড্রাই রান — ধাপে ধাপে দেখি

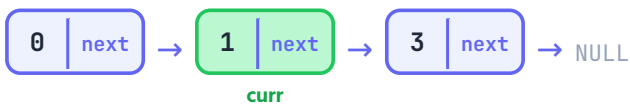
ধাপ 1 delete(2): curr = 0 থেকে শুরু। curr->next->val = 1 $\neq$ 2 $\rightarrow$ এগোও



ধাপ 2 curr = 1। curr->next->val = 2 $\rightarrow$ পেয়েছি! থামো



ধাপ 3 curr->next = curr->next->next $\rightarrow$ 1 এখন সরাসরি 3-কে দেখায়। 2 শিকল থেকে বাদ!



এজ কেস (এগুলো ভুলবে না)

- খালি list-এ delete $\rightarrow$ কিছু করবে না
- head মুছতে হলে আলাদা কেস
- মান না থাকলে $\rightarrow$ curr শেষে পৌঁছায়, কিছু বাদ যায় না

ইন্টারভিউতে যা বলবে

- "Dummy node" কৌশল বলো: head-এর আগে একটা নকল node রাখলে head মোছার আলাদা কেস লাগে না — কোড পরিষ্কার হয়।

এরপরের কোডগুলোর জন্য

পরের প্রশ্নগুলোতে ধরে নিচ্ছি `ListNode` class আছে, আর দুইটা helper: `buildList([1,2,3])` (array থেকে list বানায়) এবং `listToString($head)` ("1 -> 2 -> 3 -> NULL" রিটার্ন করে)।

PHP — helper

```
function buildList(array $values): ?ListNode {
    $dummy = new ListNode(0);
    $tail = $dummy;
    foreach ($values as $v) {
        $tail->next = new ListNode($v);
        $tail = $tail->next;
    }
    return $dummy->next;
}

function listToString(?ListNode $head): string {
    $out = [];
    while ($head !== null) { $out[] = $head->val; $head = $head->next; }
    return implode(" -> ", $out) . " -> NULL";
}

echo listToString(buildList([1, 2, 3])); // 1 -> 2 -> 3 -> NULL
```

প্রশ্ন 4.2

Reverse Linked List

Linked list উল্টে দাও

Easy

LeetCode 206

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

সবচেয়ে বেশি জিজ্ঞেস করা linked list প্রশ্ন। তিনটা pointer (prev, curr, next) নিয়ে কাজ করা শেখায়। এটা না দেখে লিখতে পারা বাধ্যতামূলক।

সমস্যাটা কী?

একটা singly linked list-এর head দেওয়া আছে। list-টা উল্টে নতুন head রিটার্ন করো।

উদাহরণ

Input: 1 → 2 → 3 → 4 → NULL

Output: 4 → 3 → 2 → 1 → NULL

Input: NULL

Output: NULL

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

একদল মানুষ সামনের জনের কাঁধে হাত রেখে লাইনে দাঁড়িয়ে। উল্টাতে হলে প্রত্যেকে একে একে হাত সরিয়ে পেছনের জনের কাঁধে রাখবে। কিন্তু হাত সরানোর আগে মনে রাখতে হবে সামনে কে ছিল — নইলে বাকি লাইন হারিয়ে যাবে! এজন্যই **next** আগে সেভ করি।

পদ্ধতি ১: Array-তে মান রেখে উল্টো ক্রমে নতুন list

সব মান array-তে নাও, তারপর পেছন থেকে নতুন list বানাও। সহজ, কিন্তু $O(n)$ extra space।

PHP

```
function reverseListArray(?ListNode $head): ?ListNode {
    $vals = [];
    for ($c = $head; $c !== null; $c = $c->next) $vals[] = $c->val;
    return buildList(array_reverse($vals));
}
echo listToString(reverseListArray(buildList([1, 2, 3, 4]))); // 4 -> 3 -> 2 -> 1 ->
NULL
```

Time: $O(n)$ Space: $O(n)$

পদ্ধতি ২: Iterative — তিন pointer (এটাই মুখস্থ রাখবে)

1. prev = NULL, curr = head
2. লুপ: **next = curr->next** (সেভ করো) → **curr->next = prev** (উল্টাও) → **prev = curr** → **curr = next**

3. শেষে prev-ই নতুন head

PHP

```
function reverseList(?ListNode $head): ?ListNode {
    $prev = null;
    $curr = $head;
    while ($curr !== null) {
        $next = $curr->next; // ১. পরেরটা সেভ করো (নইলে হারিয়ে যাবে)
        $curr->next = $prev; // ২. তীর উল্টে দাও
        $prev = $curr;       // ৩. prev এক ধাপ এগোও
        $curr = $next;       // ৪. curr এক ধাপ এগোও
    }
    return $prev;           // নতুন head
}
echo listToString(reverseList(buildList([1, 2, 3, 4]))); // 4 -> 3 -> 2 -> 1 -> NULL
```

Time: O(n)

Space: O(1)

পদ্ধতি ৩: Recursive (follow-up হিসেবে আসে)

বাকি list (head->next থেকে) উল্টাও। তারপর head->next (যেটা এখন উল্টানো অংশের শেষ) এর next = head, আর head->next = null।

PHP

```
function reverseListRec(?ListNode $head): ?ListNode {
    if ($head === null || $head->next === null) return $head; // base case
    $newHead = reverseListRec($head->next); // বাকিটা উল্টাও
    $head->next->next = $head; // আমার পরের জন এখন আমাকে দেখাবে
    $head->next = null;
    return $newHead;
}
echo listToString(reverseListRec(buildList([1, 2, 3, 4]))); // 4 -> 3 -> 2 -> 1 -> NULL
```

Time: O(n)

Space: O(n) – recursion stack

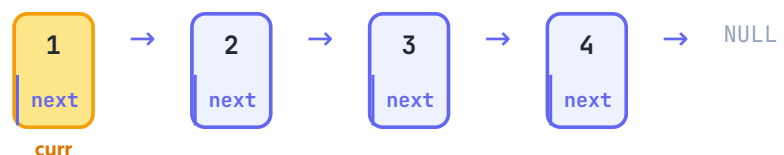
ড্রাই রান — ধাপে ধাপে দেখি

ধাপ 1 শুরু: prev = NULL, curr = 1

prev (উল্টানো অংশ)

NULL

curr (বাকি অংশ)



ধাপ 2 next=2 সেত, 1->next = NULL, prev=1, curr=2

উল্টানো অংশ



→

NULL

বাকি অংশ



→



→



→

NULL

ধাপ 3 next=3 সেত, 2->next = 1, prev=2, curr=3

উল্টানো অংশ



→



→ NULL

বাকি অংশ



→



→

NULL

ধাপ 4 next=4 সেত, 3->next = 2, prev=3, curr=4

উল্টানো অংশ



→



→



→ NULL

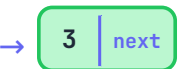
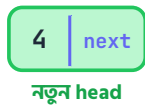
বাকি অংশ



→

NULL

ধাপ 5 4->next = 3, prev=4, curr=NULL → লুপ শেষ। prev (4) রিটার্ন!



→ NULL

এজ কেস (এগুলো ভুলবে না)

- খালি list (NULL) → NULL
- এক node → সেটাই রিটার্ন
- দুই node → ঠিক কাজ করে

ইন্টারভিউতে যা বলবে

- ৪ লাইনের ছন্দ মুখস্থ করো: "সেত, উল্টাও, prev এগোও, curr এগোও"।
- বোর্ডে ছবি ঐকে দেখাও — linked list প্রশ্নে ছবি আঁকা খুব কাজে দেয়।

প্রশ্ন 4.3

Middle of the Linked List

Linked list-এর মাঝের node খোঁজো

Easy

LeetCode 876

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Fast-Slow pointer (খরগোশ-কচ্ছপ) কৌশলের সবচেয়ে সহজ উদাহরণ। এই কৌশল cycle detection, palindrome list ইত্যাদিতে লাগে।

সমস্যাটা কী?

Linked list-এর মাঝের node রিটার্ন করো। দুইটা মাঝ থাকলে (জোড় সংখ্যক node), দ্বিতীয়টা।

উদাহরণ

Input: 1 → 2 → 3 → 4 → 5

Output: 3

Input: 1 → 2 → 3 → 4 → 5 → 6

Output: 4

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

দুইজন দৌড়াচ্ছে: কচ্ছপ প্রতি সেকেন্ডে ১ ধাপ, খরগোশ ২ ধাপ। খরগোশ যখন রাস্তার শেষে পৌঁছাবে, কচ্ছপ ঠিক মাঝখানে থাকবে!

পদ্ধতি ১: দুই পাস — আগে গোনো, পরে অর্ধেক হাঁটো

প্রথমে node সংখ্যা n গোনো। তারপর head থেকে $n/2$ ধাপ হাঁটো।

PHP

```
function middleNodeCount(?ListNode $head): ?ListNode {
    $n = 0;
    for ($c = $head; $c !== null; $c = $c->next) $n++;
    $curr = $head;
    for ($i = 0; $i < intval($n, 2); $i++) $curr = $curr->next;
    return $curr;
}
echo middleNodeCount(buildList([1, 2, 3, 4, 5]))->val; // 3
```

Time: $O(n)$ Space: $O(1)$

খারাপ না! কিন্তু list দুইবার ঘুরতে হয়।

পদ্ধতি ২: Fast-Slow Pointer — এক পাস

1. slow = fast = head

2. যতক্ষণ fast আর fast->next null না: slow ১ ধাপ, fast ২ ধাপ

3. লুপ শেষে slow-ই মাঝখান

PHP

```
function middleNode(?ListNode $head): ?ListNode {
    $slow = $head;
    $fast = $head;
    while ($fast !== null && $fast->next !== null) {
        $slow = $slow->next; // কচ্ছপ: ১ ধাপ
        $fast = $fast->next->next; // খরগোশ: ২ ধাপ
    }
    return $slow;
}

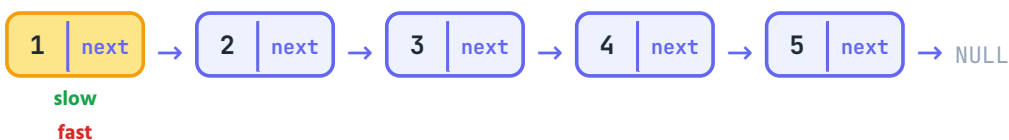
echo middleNode(buildList([1, 2, 3, 4, 5]))->val; // 3
echo " " . middleNode(buildList([1, 2, 3, 4, 5, 6]))->val; // 4
```

Time: O(n)

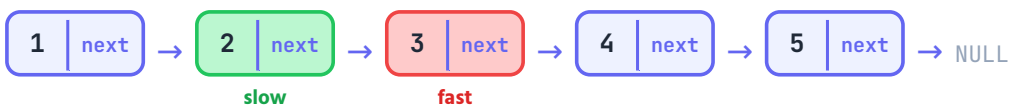
Space: O(1)

ড্রাই রান — ধাপে ধাপে দেখি

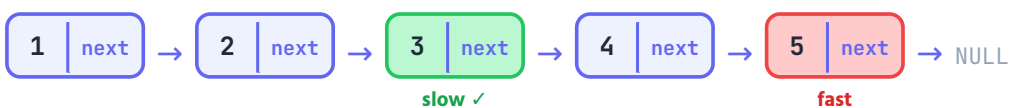
ধাপ 1 শুরু: slow = fast = 1



ধাপ 2 slow → 2, fast → 3



ধাপ 3 slow → 3, fast → 5 | fast->next = NULL → থামো। উত্তর 3



এজ কেস (এগুলো ভুলবে না)

- এক node → সেটাই মাঝ
- দুই node → দ্বিতীয়টা

ইন্টারভিউতে যা বলবে

- লুপের শর্তে `$fast !== null && $fast->next !== null` — দুটোই লাগে, নইলে null-এর ->next access করে error হবে।

প্রশ্ন 4.4

Linked List Cycle

Linked list-এ লুপ (cycle) আছে কি না

Easy

LeetCode 141

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Floyd-এর Cycle Detection (খরগোশ-কচ্ছপ) — বিখ্যাত অ্যালগরিদম। "O(1) space-এ করো" follow-up প্রায় নিশ্চিত।

সমস্যাটা কী?

Linked list-এর head দেওয়া আছে। কোনো node-এর next যদি আগের কোনো node-কে দেখায় (ফলে list কখনো শেষ হয় না), তাহলে true।

উদাহরণ

Input: 3 → 2 → 0 → -4 → (আবার 2-তে ফেরে)

Output: true

Input: 1 → 2 → NULL

Output: false

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

গোল দৌড়ের মাঠে দুইজন দৌড়াচ্ছে, একজন দ্বিগুণ গতিতে। মাঠ গোল হলে দ্রুতজন একসময় ধীরজনকে পেছন থেকে ধরে ফেলবেই। সোজা রাস্তা হলে দ্রুতজন শেষ মাথায় পৌঁছে যাবে — কখনো দেখা হবে না।

পদ্ধতি ১: Hash Set — দেখা node মনে রাখো

প্রতিটা node set-এ রাখো। কোনো node দ্বিতীয়বার এলে cycle আছে।

PHP

```
function hasCycleSet(?ListNode $head): bool {
    $seen = [];
    while ($head !== null) {
        $id = spl_object_id($head); // node-এর ইউনিক id
        if (isset($seen[$id])) return true;
        $seen[$id] = true;
        $head = $head->next;
    }
    return false;
}

$a = buildList([3, 2, 0, -4]);
$a->next->next->next->next = $a->next; // -4 → 2 (cycle তৈরি)
var_dump(hasCycleSet($a));           // true
```

Time: O(n)

Space: O(n)

মান (val) না, node নিজেই রাখতে হবে — কারণ দুইটা আলাদা node-এর মান একই হতে পারে।

পদ্ধতি ২: Floyd's Algorithm — O(1) space

1. slow ১ ধাপ, fast ২ ধাপ
2. কোনো সময় slow == fast হলে cycle আছে
3. fast null-এ পৌঁছালে cycle নেই

PHP

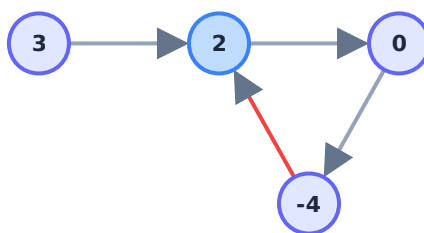
```
function hasCycle(?ListNode $head): bool {
    $slow = $head;
    $fast = $head;
    while ($fast !== null && $fast->next !== null) {
        $slow = $slow->next;
        $fast = $fast->next->next;
        if ($slow === $fast) return true;    // একই node-এ দেখা!
    }
    return false;                          // শেষ পাওয়া গেছে → cycle নেই
}
$a = buildList([3, 2, 0, -4]);
$a->next->next->next->next = $a->next;
var_dump(hasCycle($a));                  // true
var_dump(hasCycle(buildList([1, 2])));   // false
```

Time: O(n)

Space: O(1)

ভ্রাই রান — ধাপে ধাপে দেখি

লাল তীর: -4 আবার 2-কে দেখায় → cycle



ধাপ	slow	fast	মিলেছে?
শুরু	3	3	—
১	2	0	না
২	0	2 (-4 → 2)	না
৩	-4	-4 (0 → -4)	হ্যাঁ! cycle আছে

এজ কেস (এগুলো ভুলবে না)

- খালি list $\rightarrow$ false
- এক node যার next নিজেই $\rightarrow$ true
- এক node, next null $\rightarrow$ false

ইন্টারভিউতে যা বলবে

- কেন দেখা হবেই? cycle-এর ভেতরে প্রতি ধাপে fast আর slow-এর দূরত্ব ১ করে কমে — তাই একসময় ০ হবেই, "লাফিয়ে পার" হতে পারে না।

ফলো-আপ প্রশ্ন (ইন্টারভিউয়ার জিজ্ঞেস করতে পারে)

"Cycle কোথা থেকে শুরু?" (LeetCode 142) — দেখা হওয়ার পর একটা pointer head-এ ফেরত আনো, দুজনই ১ ধাপ করে চলো; যেখানে আবার দেখা হবে সেটাই cycle-এর শুরু।

প্রশ্ন 4.5

Merge Two Sorted Lists

দুইটা sorted list জোড়া লাগাও

Easy

LeetCode 21

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Dummy node কৌশল শেখায় আর merge sort-এর merge ধাপের linked list ভার্সন। খুব কমন।

সমস্যাটা কী?

দুইটা sorted linked list `list1` আর `list2` -কে একটা sorted list-এ merge করো (নতুন node না বানিয়ে, পুরনোগুলো জোড়া দিয়ে)।

উদাহরণ

Input: 1 → 2 → 4, 1 → 3 → 4

Output: 1 → 1 → 2 → 3 → 4 → 4

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

দুইটা লাইনে ছাত্রের উচ্চতা অনুযায়ী দাঁড়ানো। একটা নতুন লাইন বানাবে: দুই লাইনের সামনের দুইজনকে তুলনা করো, খাটো জনকে নতুন লাইনে পাঠাও। বারবার।

পদ্ধতি ১: সব মান array-তে, sort, নতুন list

দুই list-এর সব মান এক array-তে নাও, sort করো, নতুন list বানাও। sorted হওয়ার সুবিধা নষ্ট।

PHP

```
function mergeListsArray(?ListNode $a, ?ListNode $b): ?ListNode {
    $vals = [];
    for ($c = $a; $c !== null; $c = $c->next) $vals[] = $c->val;
    for ($c = $b; $c !== null; $c = $c->next) $vals[] = $c->val;
    sort($vals);
    return buildList($vals);
}
echo listToString(mergeListsArray(buildList([1, 2, 4]), buildList([1, 3, 4])));
```

Time: $O((n+m) \log(n+m))$ Space: $O(n+m)$

পদ্ধতি ২: Dummy node + দুই pointer

1. একটা dummy node বানাও, tail = dummy
2. দুই list-এর সামনের node তুলনা করো, ছোটটা tail->next-এ জুড়ে দাও, সেই list এগোও
3. একটা শেষ হলে অন্যটার বাকি পুরোটা জুড়ে দাও

4. dummy->next রিটার্ন

PHP

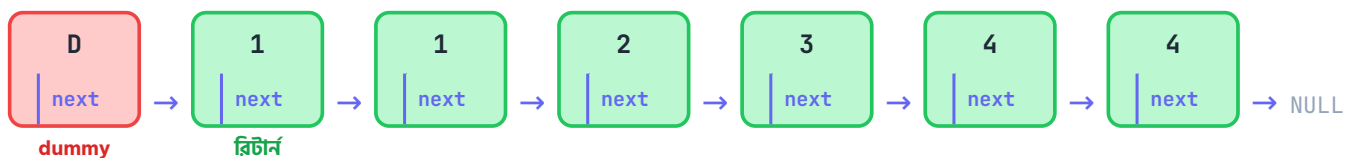
```
function mergeTwoLists(?ListNode $l1, ?ListNode $l2): ?ListNode {
    $dummy = new ListNode(0);    // নকল শুরু - head নিয়ে আলাদা চিন্তা লাগে না
    $tail = $dummy;
    while ($l1 !== null && $l2 !== null) {
        if ($l1->val <= $l2->val) {
            $tail->next = $l1;
            $l1 = $l1->next;
        } else {
            $tail->next = $l2;
            $l2 = $l2->next;
        }
        $tail = $tail->next;
    }
    $tail->next = $l1 ?? $l2;    // যেটা বাকি আছে পুরোটা জুড়ে দাও
    return $dummy->next;
}

echo listToString(mergeTwoLists(buildList([1, 2, 4]), buildList([1, 3, 4])));
// 1 -> 1 -> 2 -> 3 -> 4 -> 4 -> NULL
```

Time: $O(n + m)$ Space: $O(1)$

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ	l1-এর সামনে	l2-এর সামনে	কাকে নিলাম	ফলাফল
১	1	1	l1 ($1 \leq 1$)	$D \rightarrow 1$
২	2	1	l2	$D \rightarrow 1 \rightarrow 1$
৩	2	3	l1	$\dots \rightarrow 2$
৪	4	3	l2	$\dots \rightarrow 3$
৫	4	4	l1	$\dots \rightarrow 4$
৬	NULL	4	l2-এর বাকিটা	$\dots \rightarrow 4 \rightarrow \text{NULL}$



এজ কেস (এগুলো ভুলবে না)

- একটা list খালি $\rightarrow$ অন্যটা রিটার্ন
- দুটোই খালি $\rightarrow \text{NULL}$

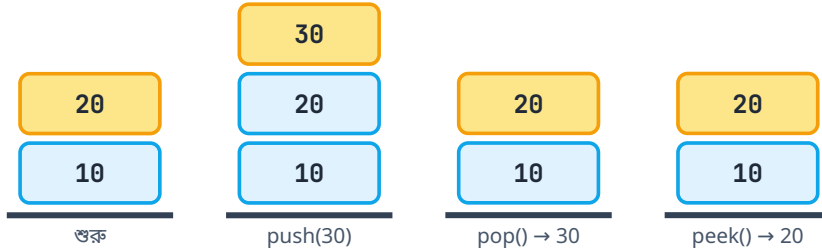
ইন্টারভিউতে যা বলবে

- Dummy node কেন? "প্রথম node কোনটা হবে" তা নিয়ে আলাদা if লাগে না। Linked list প্রশ্নে এই কৌশল বারবার কাজে লাগে।

অধ্যায় ২: Stack, Queue ও Deque

Stack — LIFO (Last In, First Out)

যে সবশেষে ঢুকেছে, সে সবার আগে বের হয়। শুধু উপর (top) থেকে যোগ (push) আর বাদ (pop) হয়।



বাস্তব উদাহরণ

প্লেটের স্ট্যুপ — উপরে রাখো, উপর থেকেই নাও। এছাড়া: Ctrl+Z (undo), browser-এর back বাটন, ফাংশন কল (call stack), bracket মেলানো।

Queue — FIFO (First In, First Out)

যে আগে ঢুকেছে, সে আগে বের হয়। পেছনে যোগ (enqueue), সামনে থেকে বাদ (dequeue)।



বাস্তব উদাহরণ

ব্যাংকের লাইন — আগে এলে আগে সেবা। এছাড়া: প্রিন্টার queue, Laravel job queue, BFS (দিন ৬)।

Deque — Double-Ended Queue

দুই মাথা থেকেই যোগ আর বাদ করা যায় — stack আর queue দুটোর কাজই করে। Sliding window maximum-এর মতো প্রস্নে লাগে।

সামনে ও পেছনে — দুই দিকেই push/pop $O(1)$



PHP-তে ব্যবহার

PHP

```
// ---- Stack: সাধারণ array দিয়েই যথেষ্ট----
$stack = [];
array_push($stack, 1);          // বা $stack[] = 1;    O(1)
$stack[] = 2;
echo array_pop($stack) . " "; // 2    O(1)
echo end($stack);              // 1    peek (top দেখা)
echo empty($stack) ? " খালি" : " খালি না";

// ---- Queue: SplQueue ব্যবহার করো ----
$q = new SplQueue();
$q->enqueue("A");              // O(1)
$q->enqueue("B");
echo "\n" . $q->dequeue();      // A    O(1)
echo " size=" . count($q);      // 1

// ---- Deque: SplDoublyLinkedList ----
$dq = new SplDoublyLinkedList();
$dq->push(2); $dq->unshift(1); $dq->push(3); // [1, 2, 3]
echo "\n" . $dq->shift() . " " . $dq->pop(); // 1 3
```

Queue-এর জন্য array_shift() কেন খারাপ?

`array_shift()` প্রথম element বাদ দিয়ে বাকি সবার index নতুন করে সাজায় — **O(n)**। n বার dequeue করলে মোট **O(n²)**! তাই queue-এর জন্য `SplQueue` (**O(1)**) ব্যবহার করো, আর ইন্টারভিউতে এই কারণটা বলো।

	Stack	Queue	Deque
নিয়ম	LIFO	FIFO	দুই দিক
যোগ	push (top)	enqueue (rear)	push / unshift
বাদ	pop (top)	dequeue (front)	pop / shift
PHP	array + array_pop	SplQueue	SplDoublyLinkedList
সব অপারেশন	O(1)	O(1)	O(1)

প্রশ্ন 4.6

Valid Parentheses

Bracket-গুলো ঠিকভাবে খোলা-বন্ধ হয়েছে কি না

Easy

LeetCode 20

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Stack-এর সবচেয়ে বিখ্যাত প্রশ্ন। কোড এডিটর, compiler, JSON parser — সবাই এই যুক্তি ব্যবহার করে।

সমস্যাটা কী?

শুধু `() {} []` দিয়ে তৈরি string। Valid হবে যদি: প্রতিটা খোলা bracket একই ধরনের bracket দিয়ে বন্ধ হয়, এবং সঠিক ক্রমে বন্ধ হয়।

উদাহরণ

Input: `"()[]{}"`Output: `true`Input: `"{[()]}"`Output: `true`Input: `"([)]"`Output: `false` // ক্রম ভুলInput: `"(("`Output: `false` // বন্ধ হয়নি

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

রাশিয়ান পুতুল (একটার ভেতর আরেকটা)। যে পুতুলটা সবশেষে খুলেছো, সেটাই সবার আগে বন্ধ করতে হবে। "সবশেষে খোলা, সবার আগে বন্ধ" = LIFO = Stack!

পদ্ধতি ১: বারবার `"()" "[]" "{}"` মুছে ফেলা

যতক্ষণ string-এ কোনো খালি জোড়া আছে, মুছে ফেলো। শেষে string খালি হলে valid।

PHP

```
function isValidReplace(string $s): bool {
    do {
        $before = $s;
        $s = str_replace(["()", "[]", "{}"], "", $s);
    } while ($s !== $before); // আর কিছু না বদলানো পর্যন্ত
    return $s === "";
}
var_dump(isValidReplace("{[()]}" )); // true
var_dump(isValidReplace("([)]" )); // false
```

Time: $O(n^2)$ Space: $O(n)$

পদ্ধতি ২: Stack

1. খোলা bracket পেলে stack-এ push
2. বন্ধ bracket পেলে: stack খালি বা top মিলছে না → false; মিললে pop
3. শেষে stack খালি হলে true (সব খোলা বন্ধ হয়েছে)

PHP

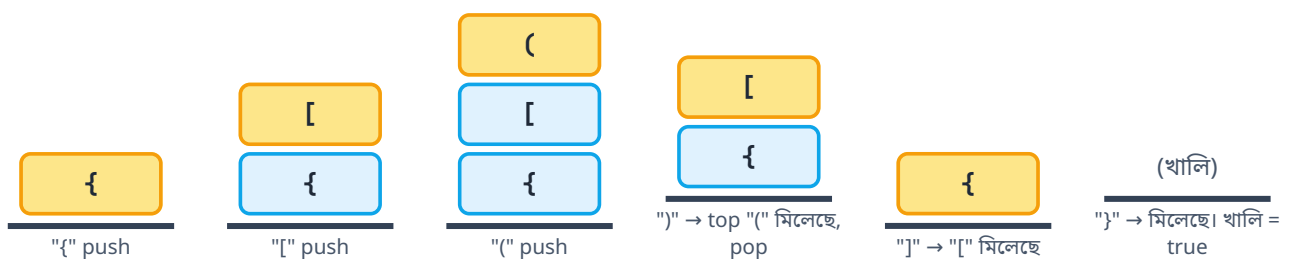
```
function isValid(string $s): bool {
    $pairs = [')' => '(', ']' => '[', '}' => '{']; // বন্ধ => তার খোলা
    $stack = [];
    for ($i = 0; $i < strlen($s); $i++) {
        $ch = $s[$i];
        if (isset($pairs[$ch])) { // বন্ধ bracket
            if (empty($stack) || array_pop($stack) !== $pairs[$ch]) {
                return false;
            }
        } else { // খোলা bracket
            $stack[] = $ch;
        }
    }
    return empty($stack);
}

var_dump(isValid("()[]{}")); // true
var_dump(isValid("{[]()}")); // true
var_dump(isValid("([)]")); // false
var_dump(isValid("((")); // false
```

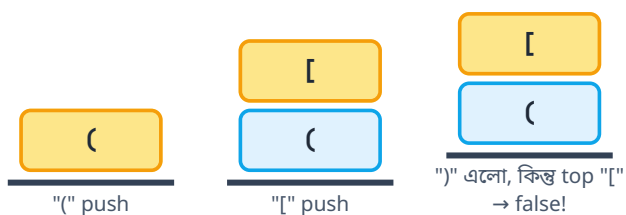
Time: $O(n)$ Space: $O(n)$

ড্রাই রান — ধাপে ধাপে দেখি

"{[()]}" (valid):



"([)]" (invalid):



এজ কেস (এগুলো ভুলবে না)

- খালি string → true
- শুধু বন্ধ ")" → stack খালি → false
- শুধু খোলা "(" → শেষে stack খালি না → false
- বিজোড় দৈর্ঘ্য → সরাসরি false (ছোট optimization)

ইন্টারভিউতে যা বলবে

- map-টা **বন্ধ → খোলা** রাখো, তাহলে বন্ধ bracket পেয়ে সরাসরি জানা যায় top-এ কী থাকা উচিত।

প্রশ্ন 4.7

Implement Queue using Stacks

দুইটা stack দিয়ে queue বানাও

Easy

LeetCode 232

কেন এটা গুরুত্বপূর্ণ?

Stack আর Queue-এর পার্থক্য সত্যিই বুঝেছো কি না যাচাই করার প্রিয় প্রশ্ন। Amortized complexity বোঝায়।

সমস্যাটা কী?

শুধু stack অপারেশন (push, pop, peek, empty) ব্যবহার করে একটা FIFO queue বানাও: `push(x)`, `pop()`, `peek()`, `empty()`।

উদাহরণ

`push(1)`, `push(2)`, `peek()` → 1, `pop()` → 1, `empty()` → false

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

দুইটা বালতি। নতুন বল "in" বালতিতে ফেলো। বের করার সময় "out" বালতি খালি থাকলে "in"-এর সব বল উল্টে "out"-এ ঢালো — এতে ক্রম উল্টে যায়, সবচেয়ে পুরনো বল উপরে চলে আসে!

দুই Stack: in আর out

1. push: সবসময় in-এ
2. pop/peek: out খালি হলে in-এর সব element pop করে out-এ push করো; তারপর out-এর top
3. প্রতিটা element সর্বোচ্চ একবার in থেকে out-এ যায় → গড়ে $O(1)$

PHP

```

class MyQueue {
    private array $in = [];
    private array $out = [];

    public function push(int $x): void {
        $this->in[] = $x;
    }
    public function pop(): int {
        $this->moveIfNeeded();
        return array_pop($this->out);
    }
    public function peek(): int {
        $this->moveIfNeeded();
        return end($this->out);
    }
    public function empty(): bool {
        return empty($this->in) && empty($this->out);
    }
    private function moveIfNeeded(): void {
        if (empty($this->out)) {           // out খালি হলেই শুধু ঢালো
            while (!empty($this->in)) {
                $this->out[] = array_pop($this->in);
            }
        }
    }
}

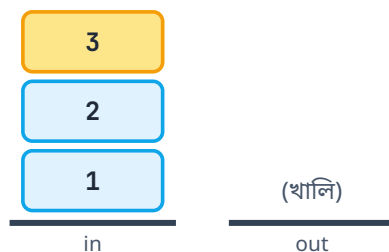
$q = new MyQueue();
$q->push(1); $q->push(2); $q->push(3);
echo $q->peek();           // 1
echo " " . $q->pop();       // 1
$q->push(4);
echo " " . $q->pop();       // 2
var_dump($q->empty());     // false

```

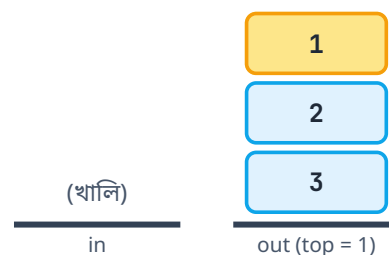
Time: push $O(1)$, pop/peek amortized $O(1)$ Space: $O(n)$

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ 1 push 1, 2, 3 → সব in-এ



ধাপ 2 pop(): out খালি → in থেকে সব ঢালো (ক্রম উল্টে যায়)



ধাপ 3 out থেকে pop $\rightarrow$ 1 (সবচেয়ে আগে ঢোকা!) FIFO ঠিক আছে



ধাপ 4 push 4 $\rightarrow$ in-এ যায়। pop() $\rightarrow$ out খালি না, তাই সরাসরি 2



ইন্টারভিউতে যা বলবে

- "Amortized $O(1)$ " ব্যাখ্যা করো: কোনো একটা pop $O(n)$ হতে পারে, কিন্তু প্রতিটা element জীবনে মাত্র একবার in $\rightarrow$ out যায়, তাই n অপারেশনে মোট $O(n) \rightarrow$ গড়ে $O(1)$ ।

থিওরি প্রশ্নোত্তর (ইন্টারভিউ স্টাইল)

প্রতিটা উত্তর ইন্টারভিউতে ৩০-৬০ সেকেন্ডে বলার মতো করে লেখা। মুখস্থ না করে নিজের ভাষায় বলার প্র্যাকটিস করো।

প্রশ্ন 1: Array আর Linked List-এর পার্থক্য কী? কখন কোনটা?

Array মেমোরিতে পাশাপাশি থাকে — index দিয়ে $O(1)$ access, কিন্তু শুরু/মাবে insert-delete $O(n)$ । Linked list node-গুলো pointer দিয়ে জোড়া — access $O(n)$, কিন্তু জানা জায়গায় insert-delete $O(1)$ । বেশি read/random access হলে array; ঘন ঘন শুরুতে বা মাঝে যোগ-বাদ হলে linked list।

প্রশ্ন 2: Singly আর Doubly linked list-এর পার্থক্য?

Singly-তে প্রতিটা node শুধু next চেনে, এক দিকে যাওয়া যায়। Doubly-তে next আর prev দুটোই — দুই দিকে যাওয়া যায়, আর node জানা থাকলে $O(1)$ -এ delete করা যায়; কিন্তু প্রতিটা node-এ বাড়তি pointer-এর মেমোরি লাগে।

প্রশ্ন 3: Stack আর Queue-এর পার্থক্য? বাস্তব উদাহরণ দাও।

Stack LIFO — শেষে যে ঢোকে সে আগে বের হয় (undo, browser back, function call stack)। Queue FIFO — আগে যে ঢোকে সে আগে বের হয় (প্রিন্টার queue, Laravel job queue, BFS)। দুটোরই মূল অপারেশন $O(1)$ ।

প্রশ্ন 4: Call Stack কী?

প্রোগ্রাম যখন একটা ফাংশন কল করে, তার লোকাল ভেরিয়েবল আর ফেরার ঠিকানা একটা stack-এ push হয়; ফাংশন শেষ হলে pop হয়। Recursion খুব গভীরে গেলে এই stack ভরে যায় — "stack overflow" error।

প্রশ্ন 5: PHP-তে Queue কীভাবে বানাবে? array_shift কেন নয়?

SplQueue ব্যবহার করবো — enqueue/dequeue $O(1)$ । array_shift প্রতিবার বাকি সব element-এর index পুনর্বিন্যাস করে, $O(n)$ — তাই বড় queue-তে ধীর।

প্রশ্ন 6: Deque কী? কোথায় লাগে?

Double-ended queue — দুই মাথা থেকেই $O(1)$ -এ যোগ ও বাদ। Stack আর queue দুটোর কাজই করে। Sliding window maximum, palindrome চেক, undo-redo-তে কাজে লাগে।

প্রশ্ন 7: Fast-slow pointer কী? কোথায় ব্যবহার করে?

দুইটা pointer, একটা ১ ধাপ আর আরেকটা ২ ধাপ করে চলে। Linked list-এর মাঝখান খোঁজা, cycle আছে কি না (Floyd), শেষ থেকে n -তম node খোঁজায় ব্যবহার হয় — সবই $O(1)$ extra space-এ।

প্রশ্ন 8: LRU Cache কীভাবে বানাবে?

Hash map + doubly linked list। Map key থেকে সরাসরি node-এ নিয়ে যায় ($O(1)$ lookup)। List ব্যবহারের ক্রম রাখে — ব্যবহার হলে node সামনে আনি, জায়গা ফুরালে পেছনের (সবচেয়ে পুরনো) node বাদ দিই। সব অপারেশন $O(1)$ ।

প্র্যাকটিস প্রবলেম (নিজে সলভ করো)

আগে নিজে ২০ মিনিট চেষ্টা করো। আটকে গেলে হিন্ট দেখো। তারপরও না পারলে সলিউশন দেখে, পরদিন আবার নিজে লেখো।

1. Remove Nth Node From End of List (Medium · LeetCode 19)

শেষ থেকে n -তম node মুছে ফেলো, এক পাসে।

হিন্ট: Dummy node। fast-কে আগে $n+1$ ধাপ এগিয়ে দাও, তারপর দুজন একসাথে চলো; fast null হলে slow->next মুছে দাও।

2. Palindrome Linked List (Easy · LeetCode 234)

$1 \rightarrow 2 \rightarrow 2 \rightarrow 1$ palindrome কি না।

হিন্ট: সহজ: মানগুলো array-তে নিয়ে two pointer। Optimized: মাঝখান খাঁজো, দ্বিতীয় অর্ধেক উল্টাও, তুলনা করো।

3. Min Stack (Medium · LeetCode 155)

push, pop, top আর getMin() — সব $O(1)$ ।

হিন্ট: দুইটা stack: একটায় মান, আরেকটায় "এই পর্যন্ত সবচেয়ে ছোট"।

4. Remove Duplicates from Sorted List (Easy · LeetCode 83)

$1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 3$

হিন্ট: $\text{curr->val} == \text{curr->next->val}$ হলে $\text{curr->next} = \text{curr->next->next}$, নইলে এগোও।

5. Implement Stack using Queues (Easy · LeetCode 225)

শুধু queue দিয়ে stack।

হিন্ট: push-এর পর আগের সব element সামনে থেকে বের করে পেছনে ঢোকাও — নতুনটা সামনে চলে আসবে।

6. Daily Temperatures (Medium · LeetCode 739)

প্রতিদিনের জন্য, কতদিন পর বেশি গরম আসবে।

হিন্ট: Monotonic stack: index রাখো; আজকের তাপমাত্রা top-এর চেয়ে বেশি হলে pop করে উত্তর বসাও।

চিট শিট — দিন ৪

প্রশ্ন	কৌশল	মূল লাইন	Time / Space
Reverse List	৩ pointer	next=curr->next; curr->next=prev; prev=curr; curr=next	$O(n)$ / $O(1)$
Middle	Fast-slow	slow ১ ধাপ, fast ২ ধাপ	$O(n)$ / $O(1)$
Cycle	Floyd	slow === fast হলে true	$O(n)$ / $O(1)$
Merge Lists	Dummy node	ছোটটা tail->next-এ, শেষে বাকিটা জুড়ে দাও	$O(n+m)$ / $O(1)$
Valid Parentheses	Stack	খোলা push, বন্ধ হলে top মেলাও	$O(n)$ / $O(n)$
Queue via Stacks	in + out stack	out খালি হলে in ঢালো	amortized $O(1)$

Linked list-এর সোনালি নিয়ম

- `->next` access করার আগে সবসময় null চেক করো
- Pointer বদলানোর আগে যেটা হারিয়ে যাবে সেটা সেভ করো
- Head বদলাতে পারে এমন প্রশ্নে dummy node নাও
- কাগজে ছবি ঐকে ২-৩ node দিয়ে ড্রাই রান করো

আজকের নিজেকে যাচাই

- ☐ ListNode class আর append/delete না দেখে লিখতে পারি
- ☐ Reverse linked list ২ মিনিটে লিখতে পারি
- ☐ Floyd-এর cycle detection কেন কাজ করে বলতে পারি
- ☐ Valid Parentheses stack দিয়ে লিখতে পারি
- ☐ Stack বনাম Queue উদাহরণসহ বলতে পারি